Doc. No.: WG21/P1478R0
Date: 2019-01-20
Reply-to: Hans-J. Boehm
Email: hboehm@google.com
Authors: Hans-J. Boehm
Audience: SG1

P1478R0: Byte-wise atomic memcpy

Several prior papers have suggested mechanisms that allow for nonatomic accesses that behave like atomics in some way. There are several possible use cases. Here we focus on seqlocks which, in our experience, seem to generate the strongest demand for such a feature.

This prposal is intended to be as simple as possible. It is in principle a pure library facility that can be implemented without compiler support. We expect that practical implementations will implement the new facilities as aliases for existing `memcpy` implementations.

There have been a number of prior proposals in this space. Most recently P0690 suggested "tearable atomics". Other solutions were proposed in N3710, which suggested more complex handling for speculative nonatomics loads. This proposal is closest in title to P0603. This is arguably the simplest and narrowest proposal.

Seqlocks

A fairly common technique to implement low cost read-mostly synchronization is to protect a block of data with an atomic version or sequence number. The writer increments the sequence number to an odd value, updates the data, and then updates the sequence number again, restoring it to an even value. The reader checks the sequence number before and after reading the data; if the two sequence number values read either differ, or are odd, the data is discarded and the operation retried.

This has the advantage that data can be updated without allocation, and that readers do not modify memory, and thus don't risk cache contention. It seems to also be a popular technique for protecting data in memory shared between processes.

Seqlock readers typically execute code along the following lines:

```
do {
  seq1 = seq_no.load(memory_order_acquire);
  data = shared_data;
  atomic_thread_fence(memory_order_acquire);
  int seq2 =  seq_no.load(memory_order_relaxed);
} while (seq1 != seq2 || seq1 & 1);
use data;
```

For details, see Boehm, Can seqlocks get along with progrmming language memory models.

It is important that the sequence number reads not be reordered with the data reads. That is ensured by the initial `memory_order_acquire` load, and by the explicit fence. But fences only order atomic accesses, and the read of `shared_data` still races with updates. Thus for the fence to be effective, and to avoid the data race, the accesses access to `shared_data` must be atomic *in spite of the fact that any data read while a write is occurring will be discarded*.

In the general case, there are good semantic reasons to require that all data accesses inside such a seqlock "critical section" must be atomic. If we read a pointer `p` as part of reading the data, and then read `*p` as well, the code inside the critical section may read from a bad address if the read of `p` happened to see a half-updated pointer value. In such cases, there is probably no way to avoid reading the pointer with a conventional atomic load, and that's exactly what's desired.

However, in many cases, particularly in the multiple process case, seqlock data consists of a single trivially copyable object, and the seqlock "critical section" consists of a simple copy operation. Under normal circumstances, this could have been written using `memcpy`. But that's unacceptable here, since `memcpy` does not generate atomic accesses, and is (according to our specification anyway) susceptable to data races.

Currently to write such code correctly, we need to basically decompose such data into many small lock-free atomic subobjects, and copy them a piece at a time. Treating the data as a single large atomic object would defeat the purpose of the seqlock, since the atomic copy operation would acquire a conventional lock. Our proposal essentially adds a convenient library facility to automate this decomposition into small objects.

We propose that both the copy from `shared_data`, and the following fence be replaced by a new `atomic_source_memcpy` call.

The proposal

We propose to introduce two additional versions of `memcpy` to resolve the above issue:

`atomic_source_memcpy(void* dest, void* source, size_t count, memory_order order)` directly addresses the seqlock reader problem. Like `memcpy`, it requires that the source and destination ranges do not overlap. It also requires that `order` is `memory_order_seq_cst`, `memory_order_acquire` or `memory_order_relaxed`. (`Memory_order_seq_cst` barely makes sense here; we do not propose it as a default.)

This behaves as if:

```
for (size_t i = 0; i < count; ++i) {
  reinterpret_cast<char*>(dest)[i] =
      atomic_ref<char>(reinterpret_cast<char*>(source)[i]).load(memory_order_relaxed);
}
atomic_thread_fence(order);
```

Note that on standard hardware, it should be OK to actually perform the copy at larger than byte granularity. Copying multiple bytes as part of one operation is indistinguishable from running them so quickly that the intermediate state is not observed. In fact, we expect that existing assembly memcpy implementations will suffice when suffixed with the required fence.

The `atomic_source_memcpy` operation would introduce a data race and hence undefined behavior if the source where simultaneously updated by an ordinary `memcpy`. Similarly, we would expect undefined behavior if the writer updates the source using atomic operations of a different granularity. To facilitate correct use, we need to also provide a corresponding version of `memcpy` that updates memory using atomic byte stores.

We thus also propose `atomic_dest_memcpy(void* dest, void* source, size_t count, memory_order order)`, where `order` is `memory_order_seq_cst`, `memory_order_release` or `memory_order_relaxed`. (`Memory_order_seq_cst` again barely makes sense.) It behaves as if:

```
atomic_thread_fence(order);
for (size_t i = 0; i < count; ++i) {
  atomic_ref<char>(reinterpret_cast<char*>(dest)[i]).store(
      reinterpret_cast<char*>(source)[i], memory_order_relaxed);
}
```

Open questions

There is a question as to whether the `order` argument should be part of the interface, and if so, whether this is the right way to handle it.

Excluding the `order` argument, and requiring the programmer to explicitly write the fence simplifies this proposal further. But I believe there are convincning reasons to include it:

1. I believe it conveys the right intuition. The combination of `atomic_dest_memcpy(..., memory_order_release)` and an `atomic_source_memcpy(..., memory_order_acquire)` that reads the resulting values establishes a synchronizes_with relationship, as expected.
2. The explicit fence in the current seqlock idiom is confusing; this will usually eliminate it.
3. It is immediately clear that `atomic_source_memcpy(..., memory_order_acquire)` cannot contribute to an out-of-thin-air result, and hence there is no need to add overhead to prevent that.

Unfortunately, defining this construct in terms of an explicit fence overconstrains the hardware a bit; if the block being copied is short enough to be copied e.g. by a single ARMv8 load-acquire instruction, this would disallow that implementation, since the fence can also establish ordering in conjunction with other earlier atomic loads, while the load-acquire instruction cannot.

An alternative would be to include the `order` argument, but not to define it in terms of a fence. This is slightly more complex, but would allow the above load-acquire implementation.

The facility here is fundamentally a C level facility, making it potentially possible to include it in C as well. This would raise the same namespace issues that P0943 is trying to address, but compatability should be possible.

It is clearly possible to put a higher-level type-safe layer on top of this that copies trivially copyable objects rather than bytes. It is not completely clear which level we should standardize.